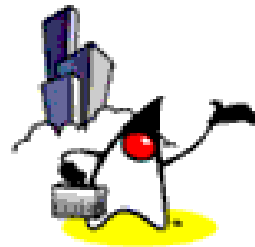
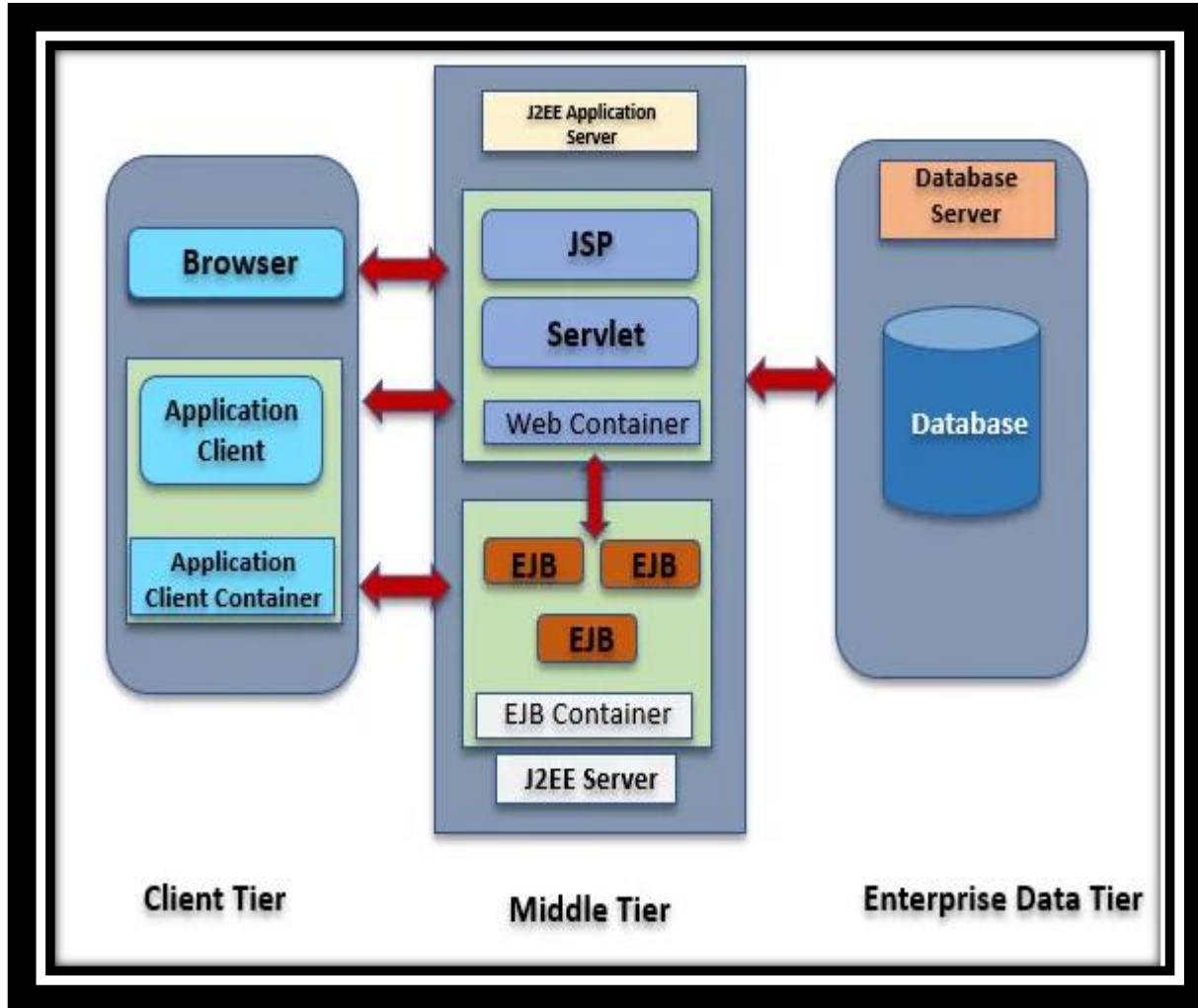




Introduction to Servlets



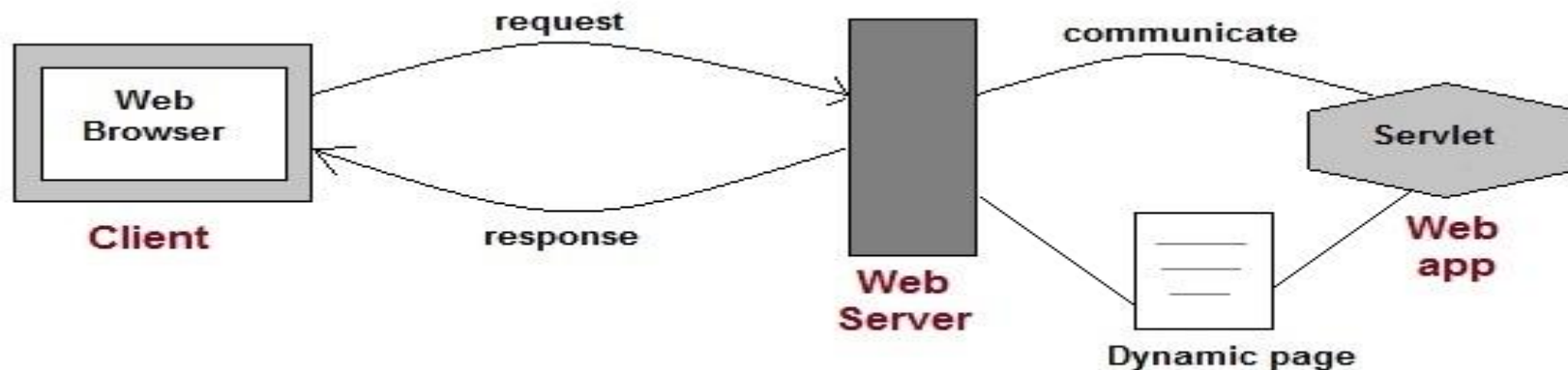
J2EE Architecture



- **Client Tier:** Client tier consists of user programs that interact with the user for request and response.
- **Middle Tier:** Middle tier usually contains enterprise beans and web services that distribute business logic for the applications.
- **Enterprise Data Tier:** Enterprise data is stored in a relational database. This tier contains containers, components and services.

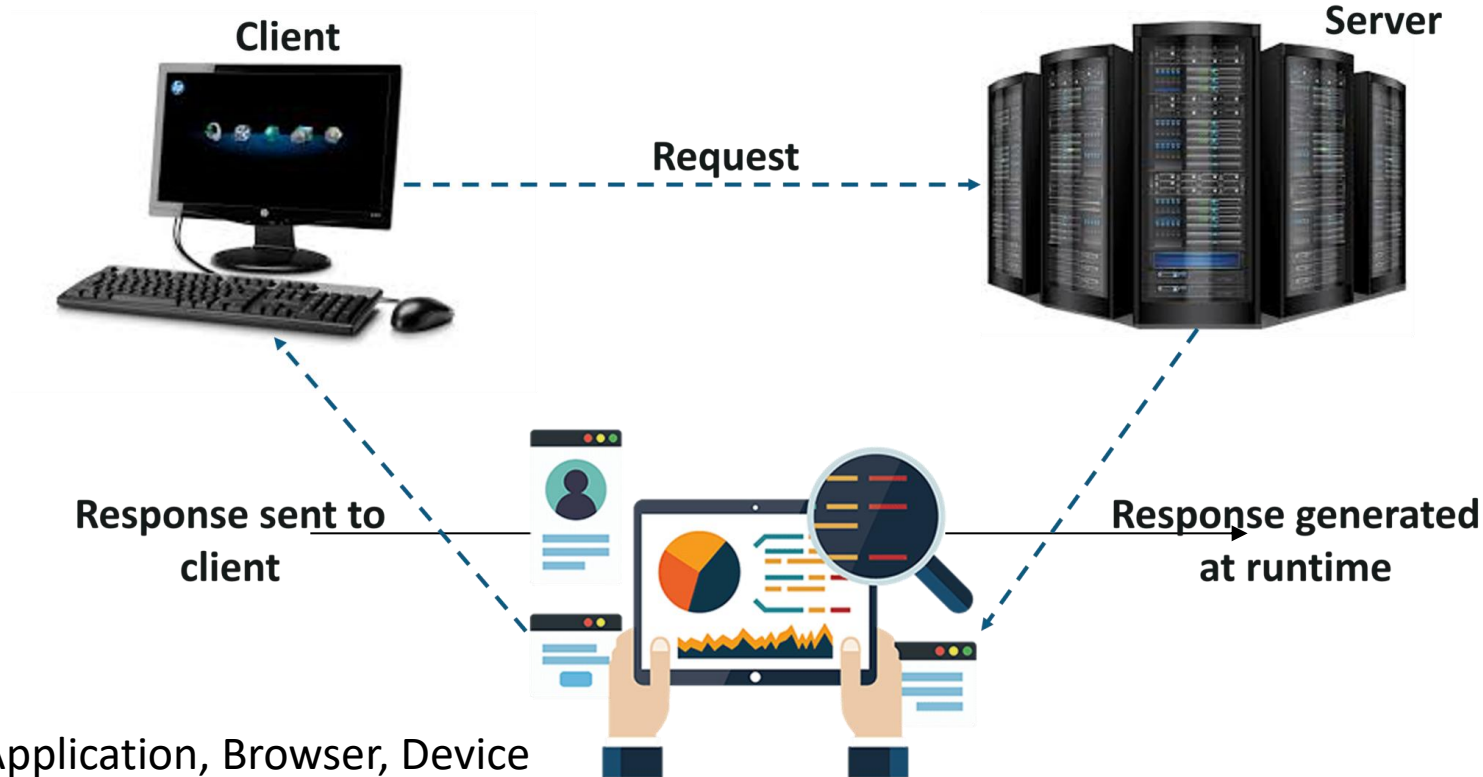
Introduction to Servlet

- Servlet Technology is used to create web applications.
- Servlet technology uses Java language to create web applications.
- Web applications are helper applications that reside at web server and build dynamic web pages.
- A dynamic page could be anything like a page that randomly chooses picture to display or even a page that displays the current time.



Introduction

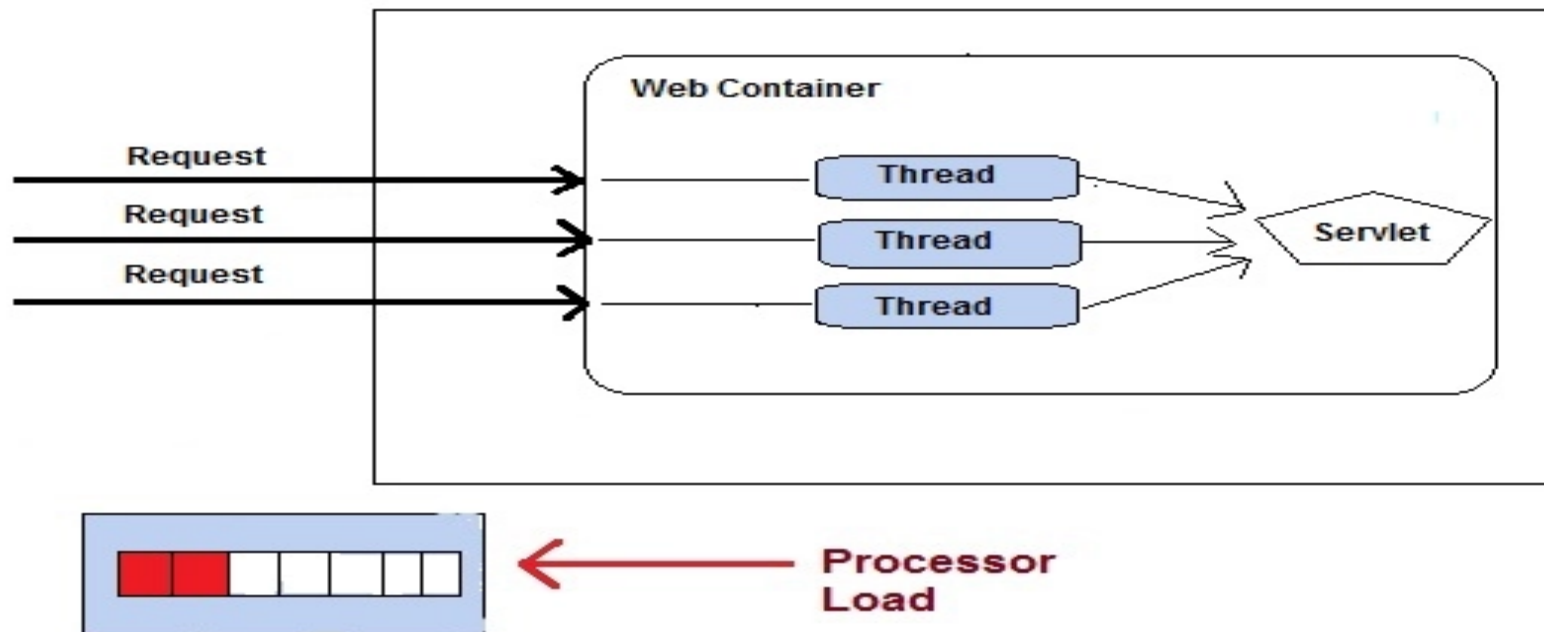
- ❑ A Servlet is a program written in Java, which runs on the server to process client requests.



- Client – Java Application, Browser, Device
- Only one instance of Servlet created for all client requests, hence save in memory.
- Servlet once initialized stays in a memory, and no need to be loaded every time a request is made.
- As Servlet Technology uses Java, web applications made using Servlet are Secured, Scalable and Robust.

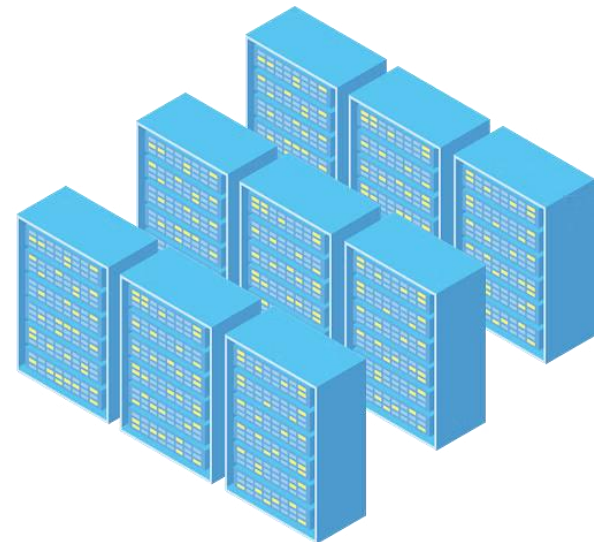
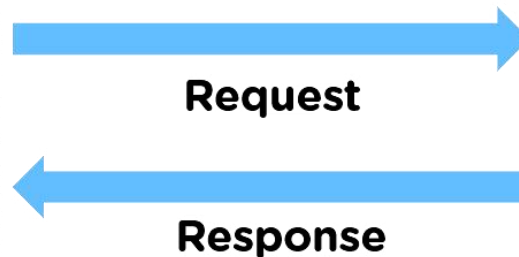
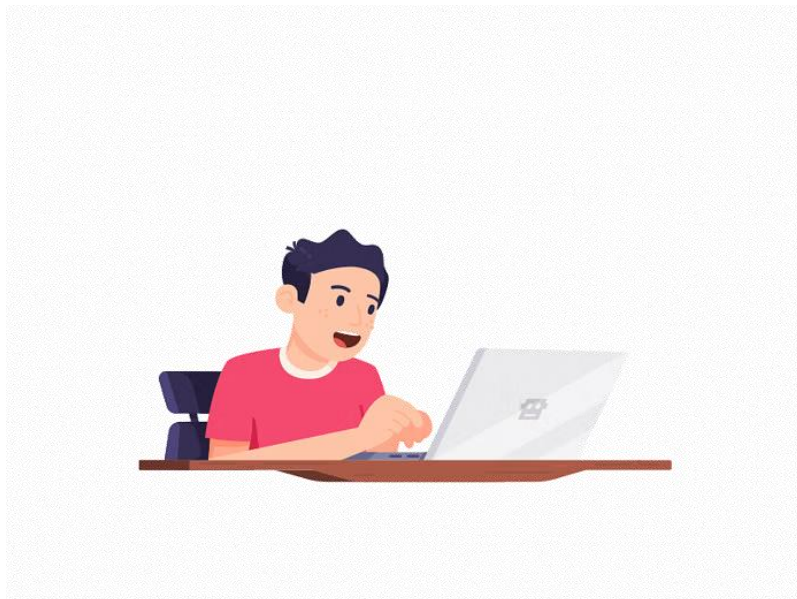
Advantages of using Servlets

- Less response time because each request runs in a separate thread.
- Servlets are scalable.
- Servlets are robust and object oriented.
- Servlets are platform independent.



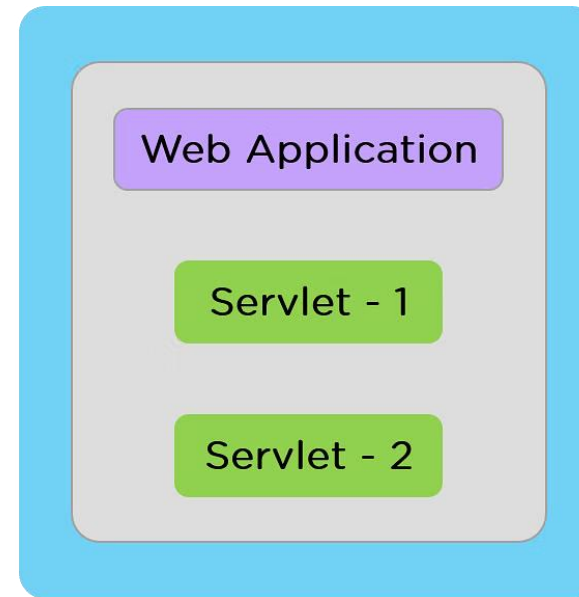
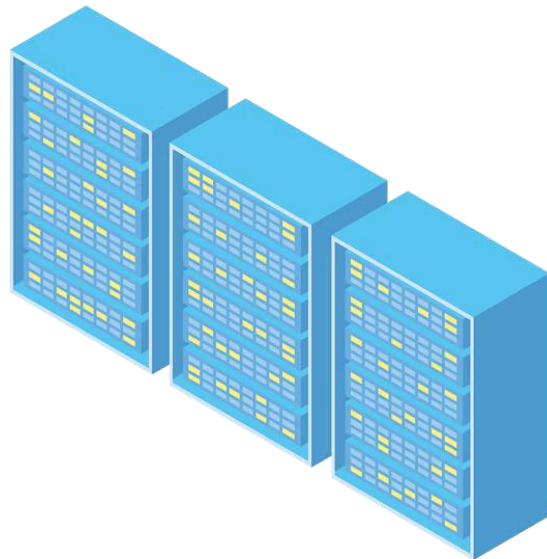
What is a Java Servlet?

- The Java Servlet, now also known as the Jakarta Servlet.
- It is a Java Server Software component, designed and deployed to enhance the Server services by upgrading their capabilities to respond to any requests through a Web API.



What is a Java Servlet?

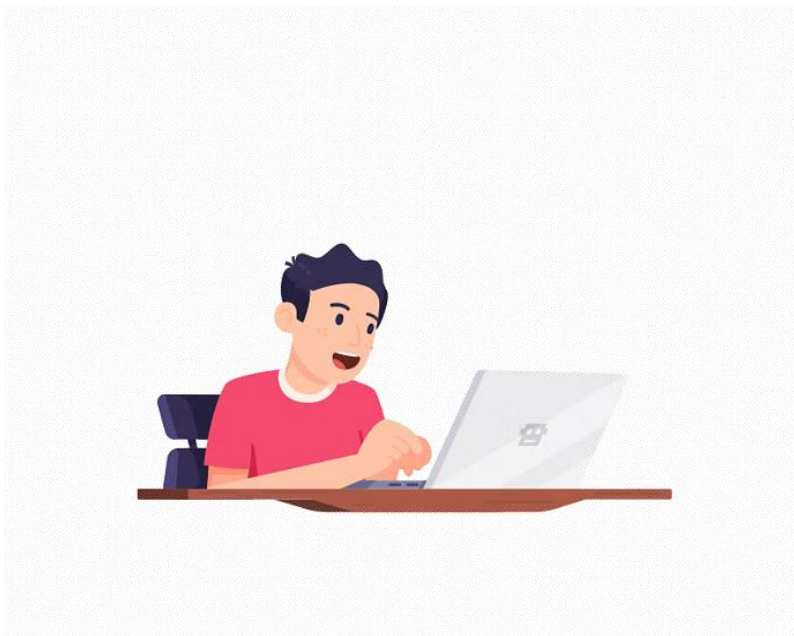
- We define a Java Servlet or Jakarta Servlet as the technology to design and deploy dynamic web pages using the Java Programming Language.
- It implements a typical servlet in the client-server architecture, and the Servlet lives on the server-side.



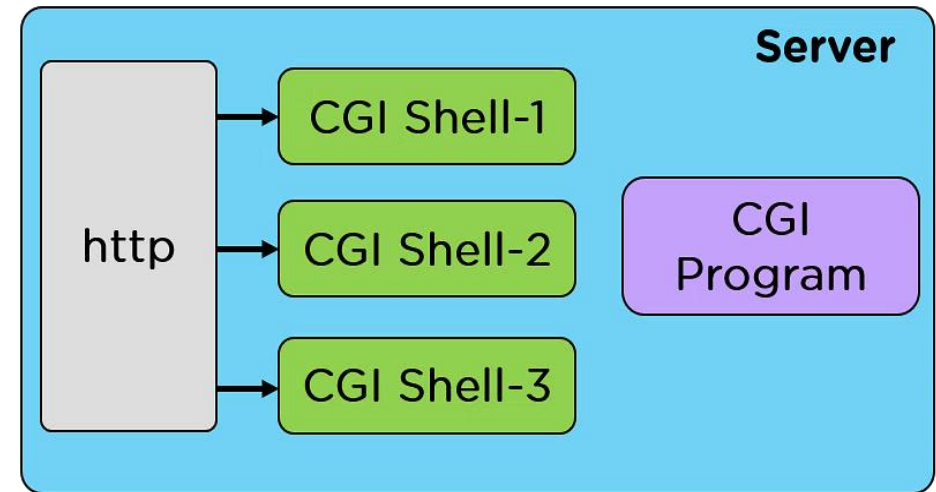
Servlet Container

CGI

- CGI stands for Common Gateway Interface.
- Like the Java Servlet, it designs the CGI to allow the webserver to call an external program, pass HTTP requests to the external program, and process it.
- It distributes the HTTP Request received amongst the CGI shells.
- In Java Servlets, it distributes the request for processing amongst the threads.



Request



Common Gateway Interface

CGI v/s Java Servlet

Java Servlets	CGI
Servlets are written and run in JVM	CGI Scripts are executable codes written in native OS
Performance is high	Performance is low
Platform independent	Hard to switch between platforms
It translates Servlets to byte-code	CGI scripts are directly executed
Servlets are portable	CGI is not portable

Servlet API

- Servlet API consists of two important packages that encapsulates all the important classes and interface, namely :
 - `javax.servlet`
 - `javax.servlet.http`

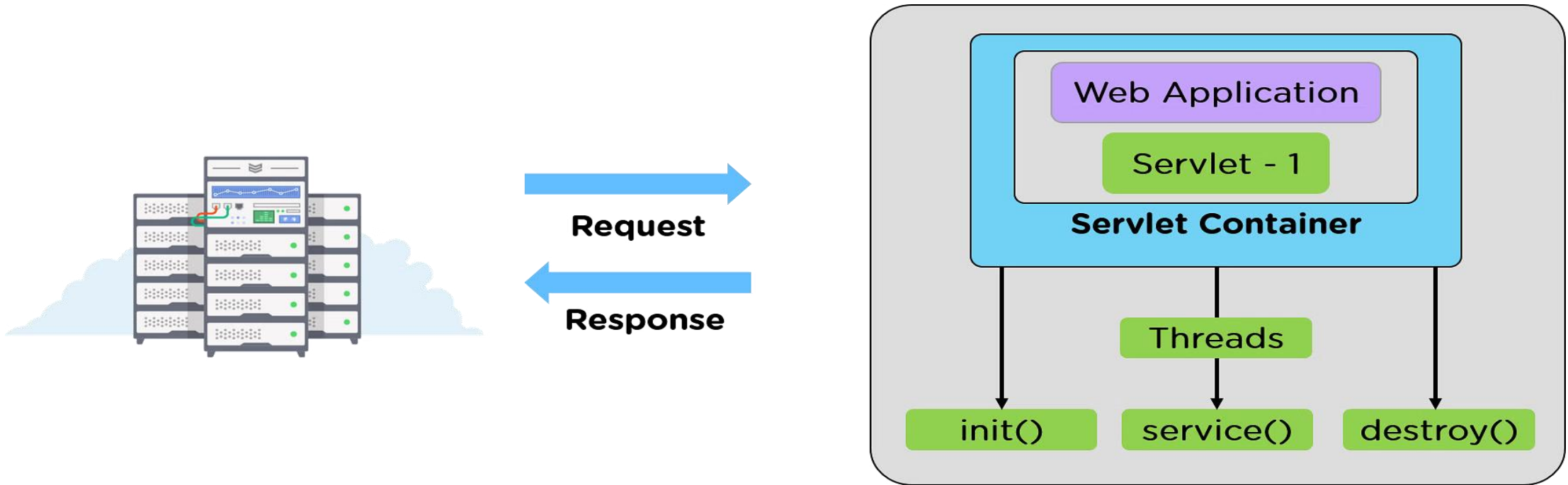
Important Classes and Interfaces of javax.servlet

INTERFACES	CLASSES
Servlet	ServletInputStream
ServletContext	ServletOutputStream
ServletConfig	ServletRequestWrapper
ServletRequest	ServletResponseWrapper
ServletResponse	ServletRequestEvent
ServletContextListener	ServletContextEvent
RequestDispatcher	ServletRequestAttributeEvent
SingleThreadModel	ServletContextAttributeEvent
Filter	ServletException
FilterConfig	UnavailableException
FilterChain	GenericServlet
ServletRequestListener	

Important Classes and Interface of javax.servlet.http

CLASSES and INTERFACES	
HttpServlet	HttpServletRequest
HttpServletResponse	HttpSessionAttributeListener
HttpSession	HttpSessionListener
Cookie	HttpSessionEvent

Java Servlet Architecture



- We can also consider the software architecture of a servlet as the life cycle of the Java Servlet.
- To write a Servlet, the user needs first to implement the Servlet Interface, directly or indirectly, using the following import command.
- `import javax.servlet.*;`

Servlet Interface

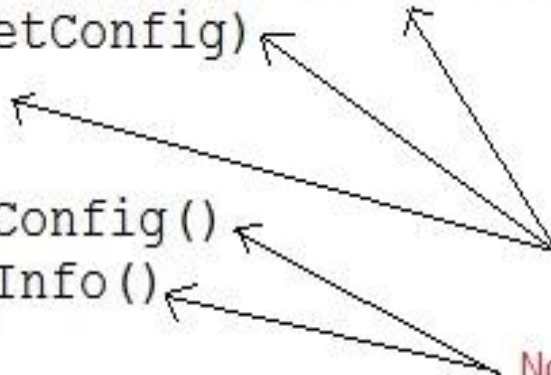
- Servlet Interface provides five methods.
- Out of these five methods, three methods are Servlet life cycle methods and rest two are non life cycle methods.

```
service(ServletRequest, ServletResponse)  
init(ServletConfig)  
destroy()
```

```
getServletConfig()  
getServletInfo()
```

Life Cycle method

Non Life Cycle method



Java Servlet Life-Cycle

The `init()` is the germinating stage of any Java Servlet. When a URL specific to a particular servlet is triggered, the `init()` method is invoked.

Another scenario when the `init()` method gets invoked is when the servers are fired up. With every server starting up, the corresponding servlets also get started, and so does the `init()` method.

One important specialty of the `init()` method is the `init()` method only gets invoked once in the entire life cycle of the Servlet, and the `init()` method will not respond to any of the user's commands.

Java Servlet Life-Cycle

The service() method is the heart of the life cycle of a Java Servlet. Right after the Servlet's initialization, it encounters the service requests from the client end.

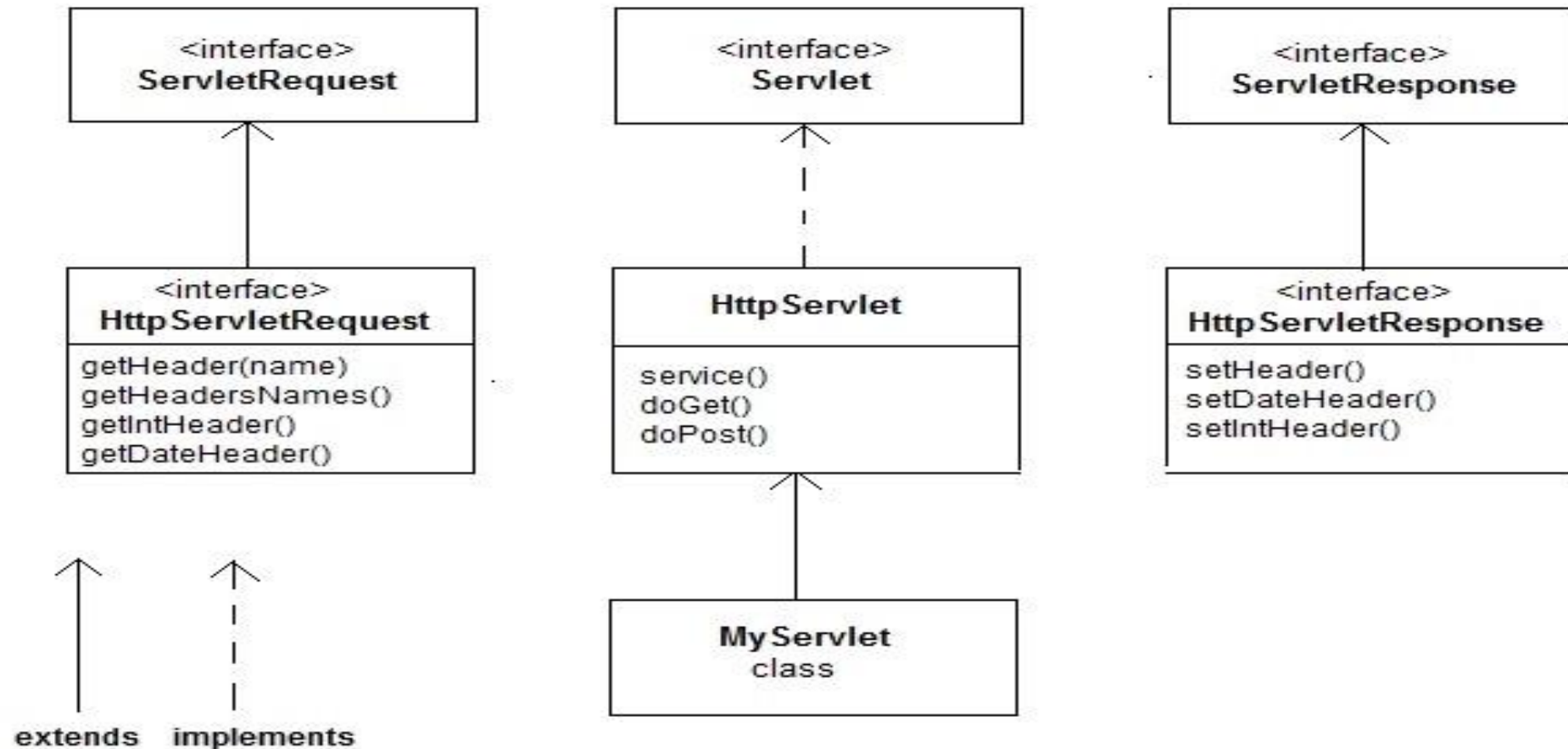
The service() method takes responsibility to check the type of request received from the client and respond accordingly by generating a new thread or a set of threads per the requirement and implementing the operation through the following methods.

- doGet() for GET
- doPut() for PUT
- doUpdate() for UPDATE
- doDelete() for DELETE

HttpServlet class

- HttpServlet is also an abstract class.
- This class gives implementation of various service() methods of Servlet interface.
- To create a servlet, we should create a class that extends HttpServlet abstract class.
- The Servlet class that we will create, must not override service() method.
- Our servlet class will override only the doGet() and/or doPost() methods.
- The service() method of HttpServlet class listens to the Http methods (GET, POST etc) from request stream and invokes doGet() or doPost() methods based on Http Method type.

HttpServlet class



Basics of HTTP Servlet 2-1

- `HttpServlet` class provides an abstract class to create an HTTP servlet.

```
protected void doGet(HttpServletRequest req,  
HttpServletResponse res);
```

`doGet()` method handles the GET request made by the client

```
protected void doPost(HttpServletRequest req,  
HttpServletResponse res);
```

`doPost()` method handles the POST request made by the client

Basics of HTTP Servlet 2-2

□ The other methods of `HttpServlet` class are:

```
protected void doDelete(HttpServletRequest req,  
HttpServletResponse res);
```

`doDelete()` method is used to delete a resource from the server

```
protected void doPut(HttpServletRequest req,  
HttpServletResponse res);
```

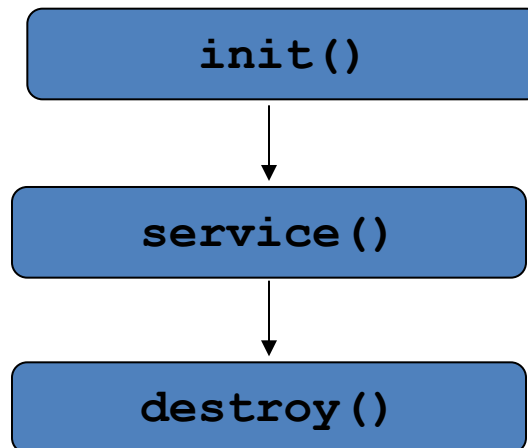
`doPut()` method is used to place a resource on the server

```
protected void doHead(HttpServletRequest req,  
HttpServletResponse res);
```

`doHead()` method handles the **HEAD** request made by the client

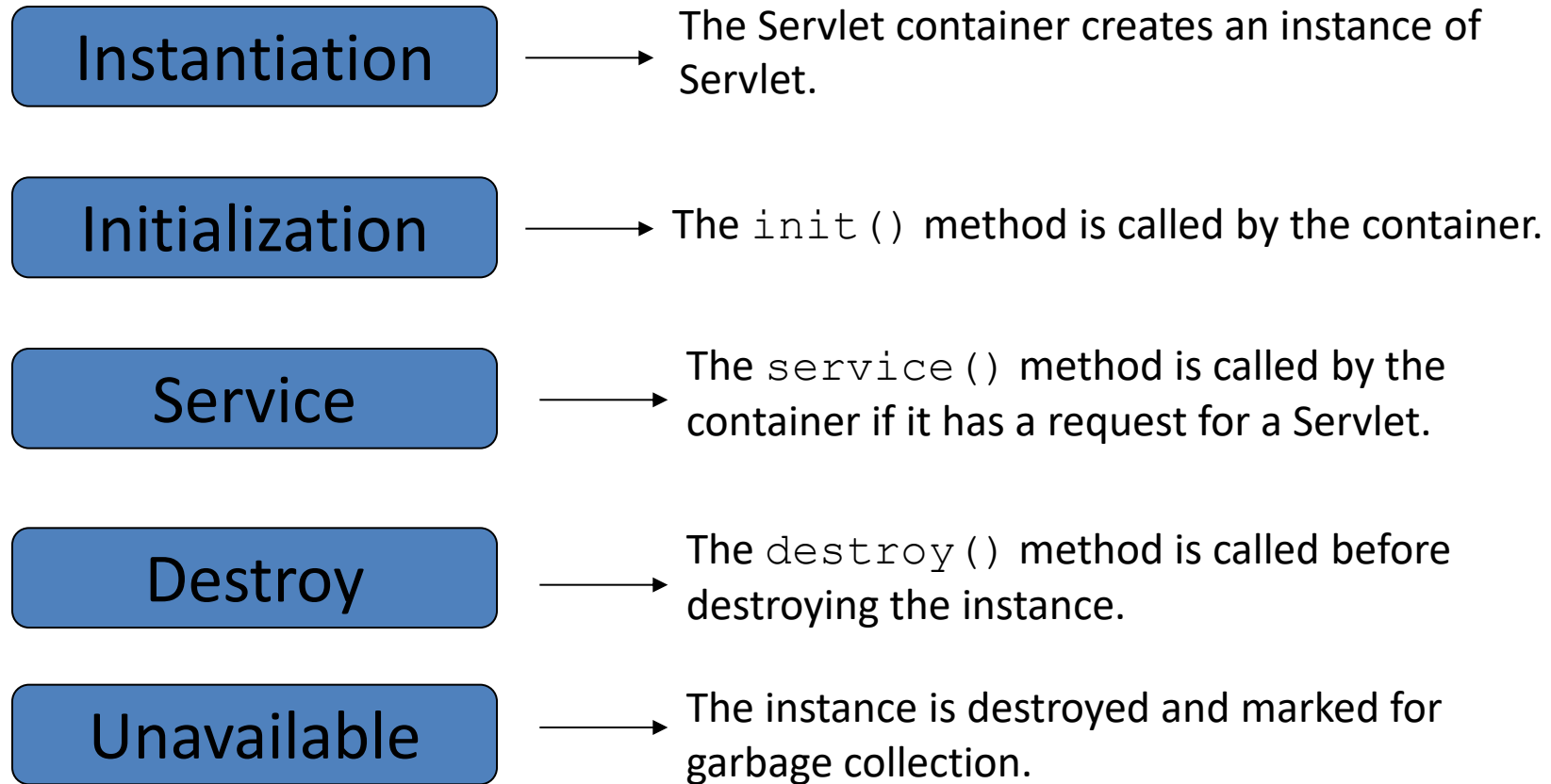
Life Cycle of a Servlet 2-1

- An instance of a Servlet is created by the Servlet container.
- The life cycle of Servlet explains when this instance was created, for how long it lived and when it expired.
- **Servlets don't have a main() method.**
- Web Container manages the life cycle of a Servlet instance.
- The 3 methods of Servlet life cycle are:



Life Cycle of a Servlet 2-2

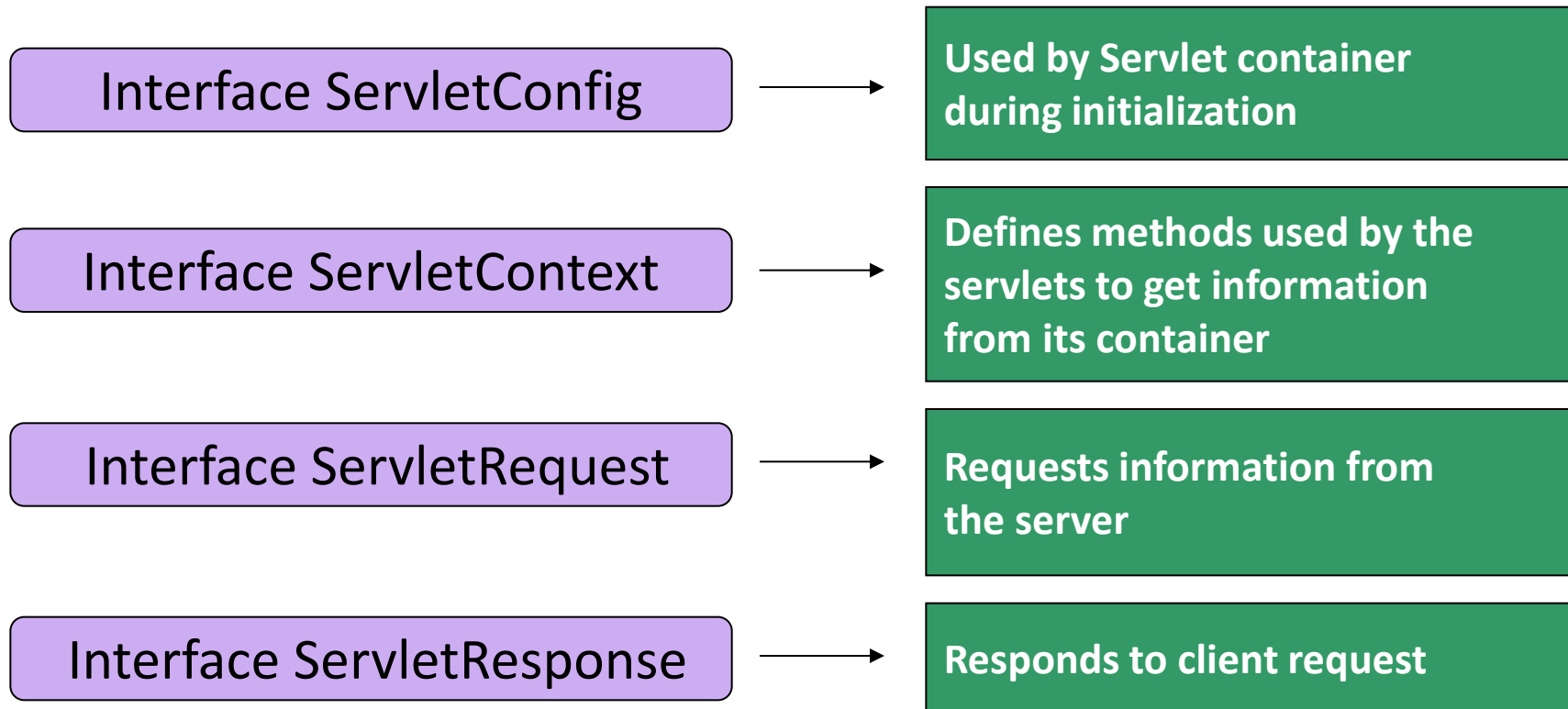
- The different stages of life cycle are:



Servlet Hierarchy 3-1

javax.servlet package

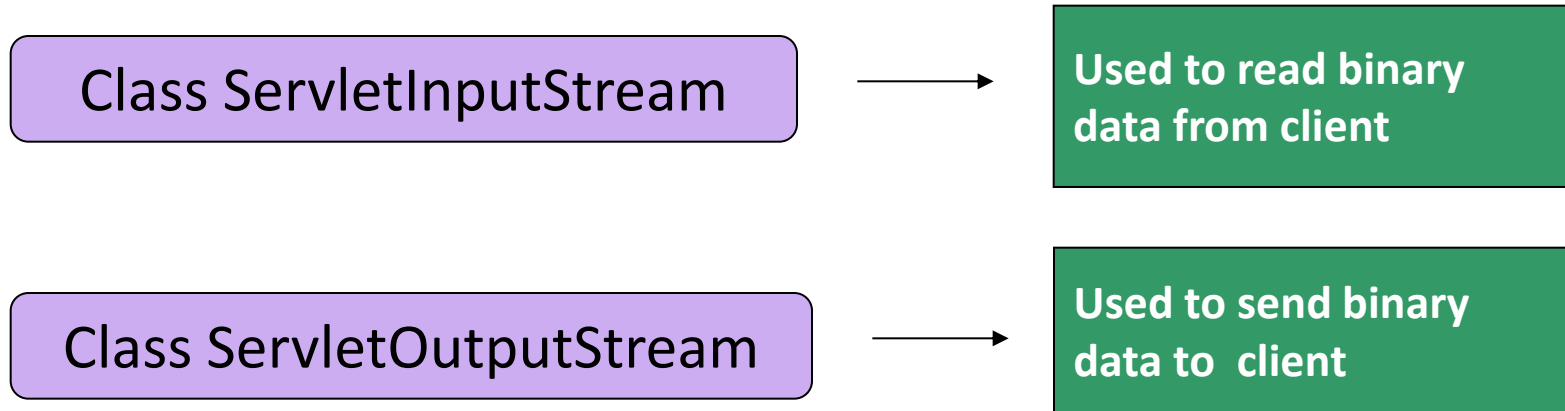
❑ The interfaces of `javax.servlet` package are:



Servlet Hierarchy 3-2

javax.servlet package

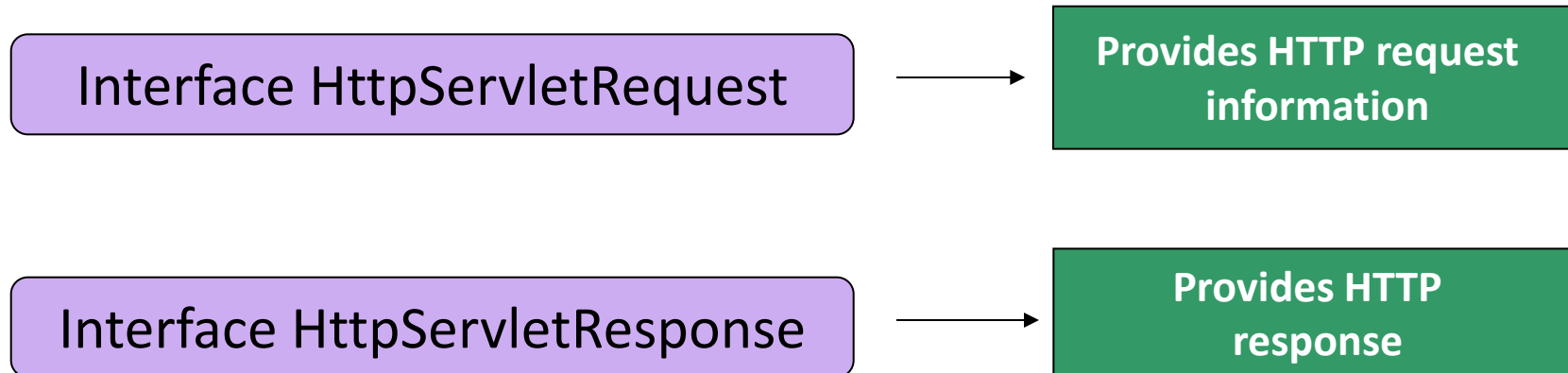
□ The classes of `javax.servlet` package are:



Servlet Hierarchy 3-3

javax.servlet.http package

□ The interfaces of `javax.servlet.http` package are:



Java Servlet Features

The Java Servlets carry over the features of Java Programming Language.

The key features offered by the Java Servlets are as follows.

Portable

Efficient

Scalable

Robust

Java Servlet Features

Portable

- As discussed above, the servlets feature the same Portable nature as the Java Programming Language.
- The Servlet program designed in one Operating System's Platform can be run in a different Operating System Platform with ease.

Efficient

- The Java Servlet, by its nature, provides an instantaneous response to the client's request.
- Also, it can be portable and perform in every environmental condition regardless of the operating system platform.

Java Servlet Features

Scalable

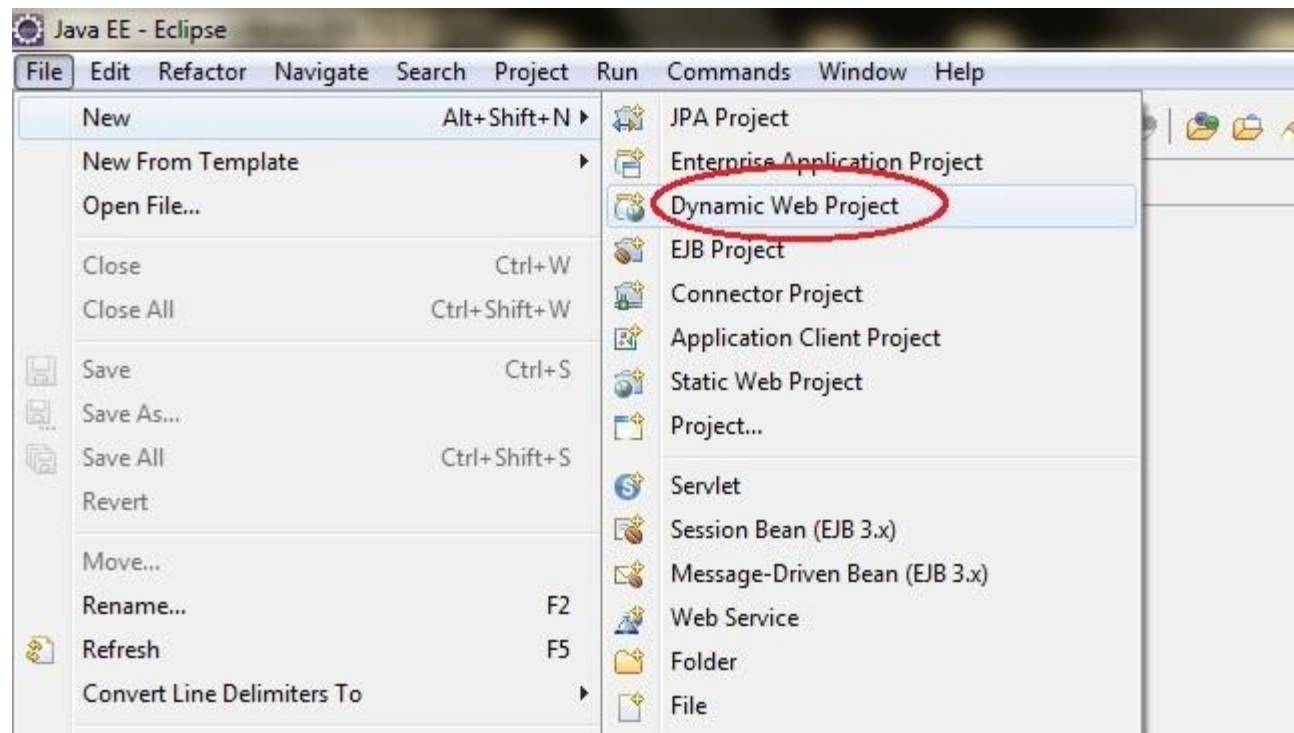
- We consider Java Servlets to be highly scalable.
- Servlets use completely lightweight threads for the processes and can simultaneously handle multiple client requests by generating various threads.

Robust

- The Java Servlets are best known for their robust operating procedures.
- The servlets extend the features of Java, which include.
 - Java Security Manager
 - Java Garbage Collector
 - Exception Handling.
- Implementation of these advanced features makes Java Servlets resilient to any security threats to a decent extent.
- Also, the garbage collector takes care of memory management and eliminates the issues related to memory management in real-time, leaving Servlets to be Robust.

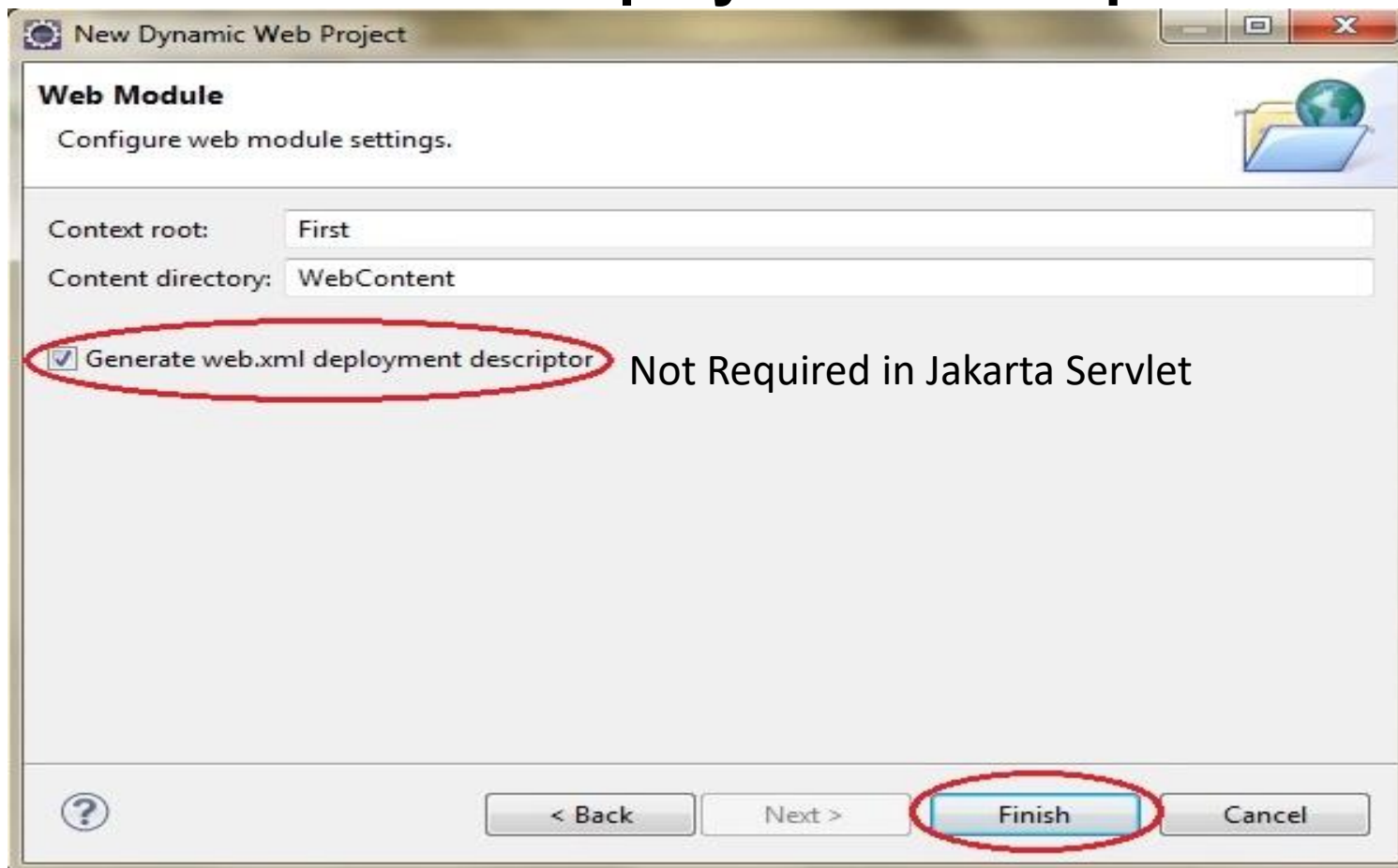
Creating First Servlet Application using Eclipse IDE

- Eclipse IDE is the most popular Java IDE used in the Industry.
- It is developed by an open source community and can be downloaded for free from Eclipse.org
- Goto **File -> New -> Dynamic Web Project**



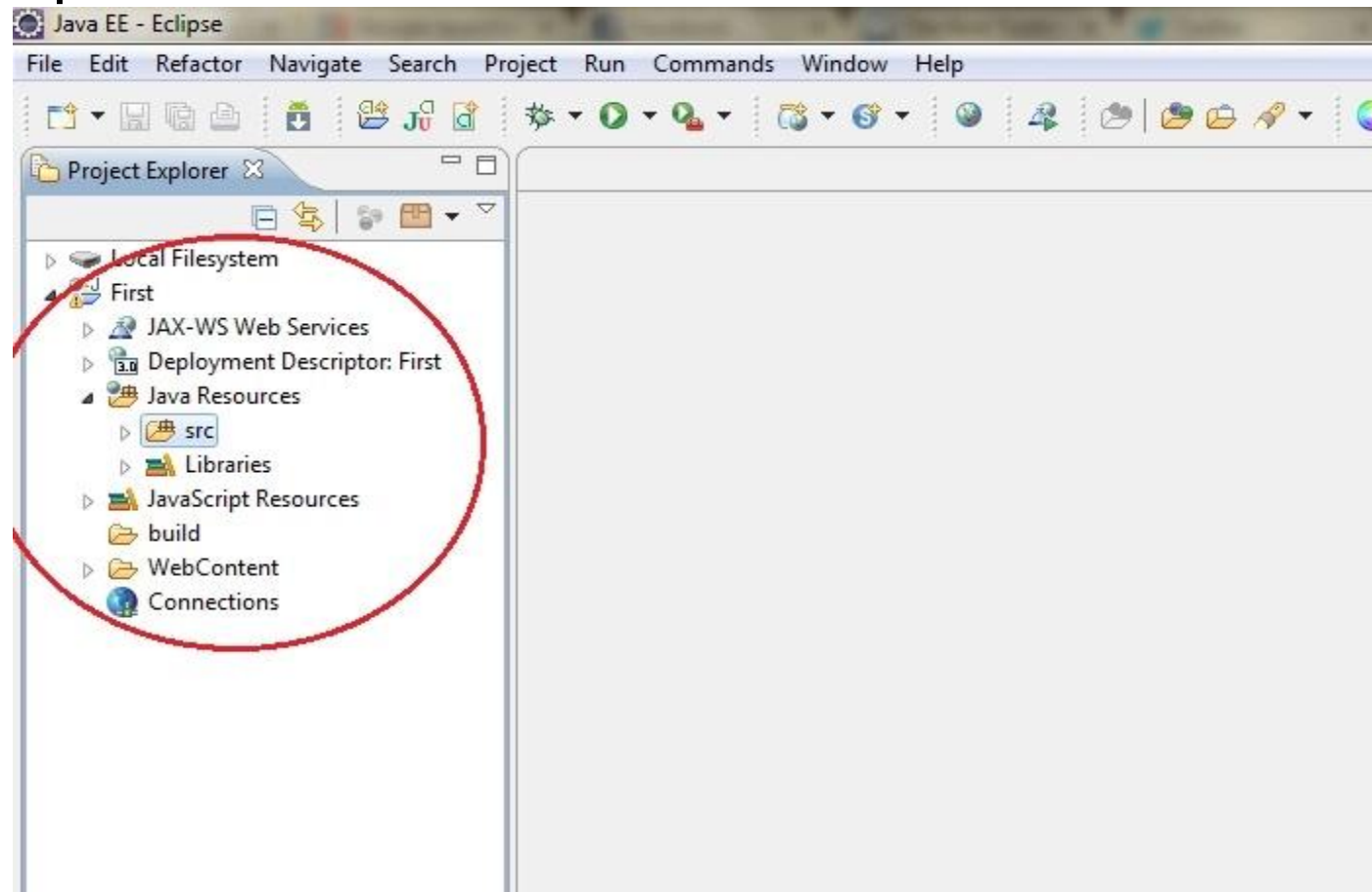
Using Eclipse IDE

- Give a Name to your Project and click Next ---- > Next
- Check **Generate web.xml Deployment Descriptor** and click **Finish**



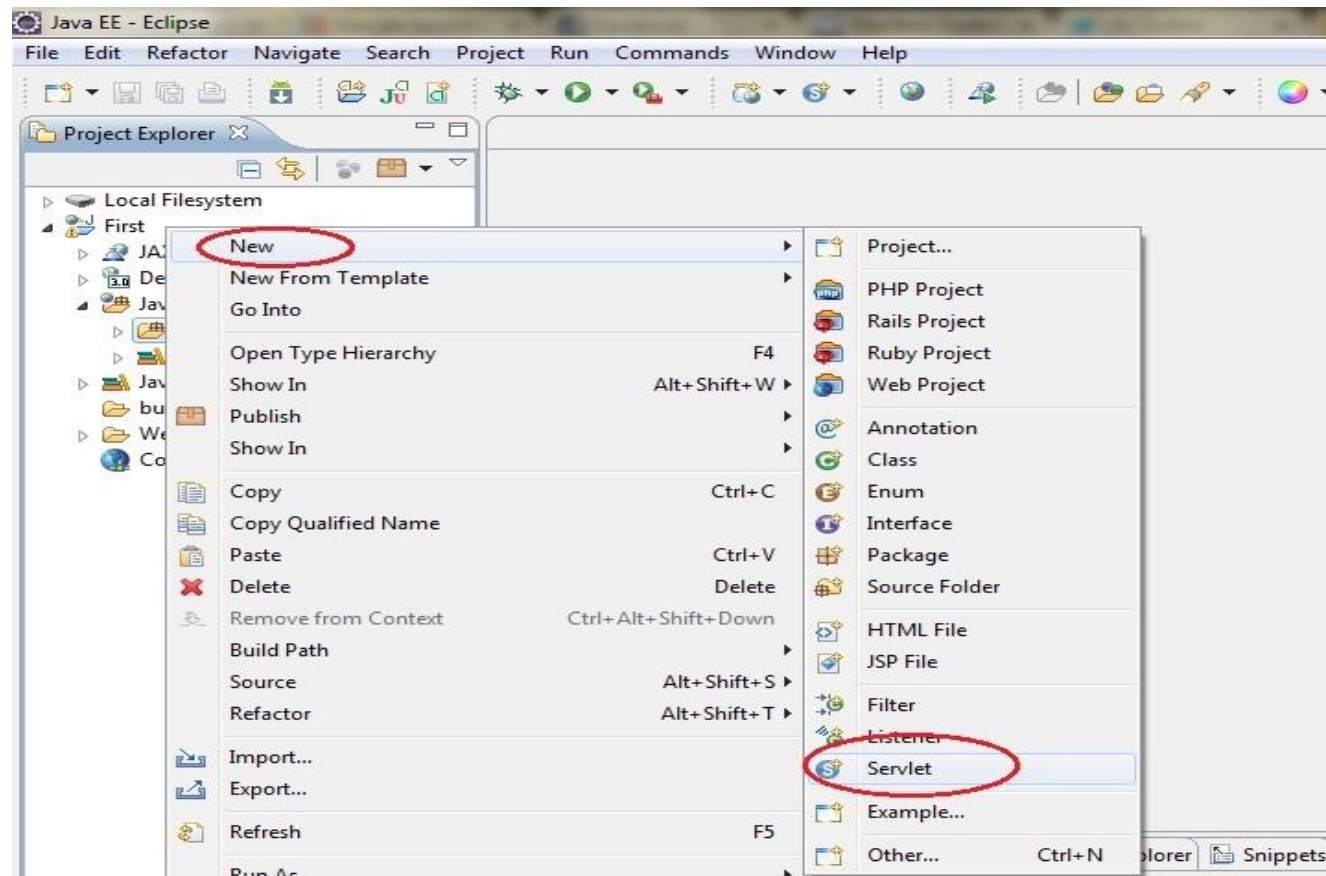
Using Eclipse IDE

- Now, the complete directory structure of your Project will be automatically created by Eclipse IDE.



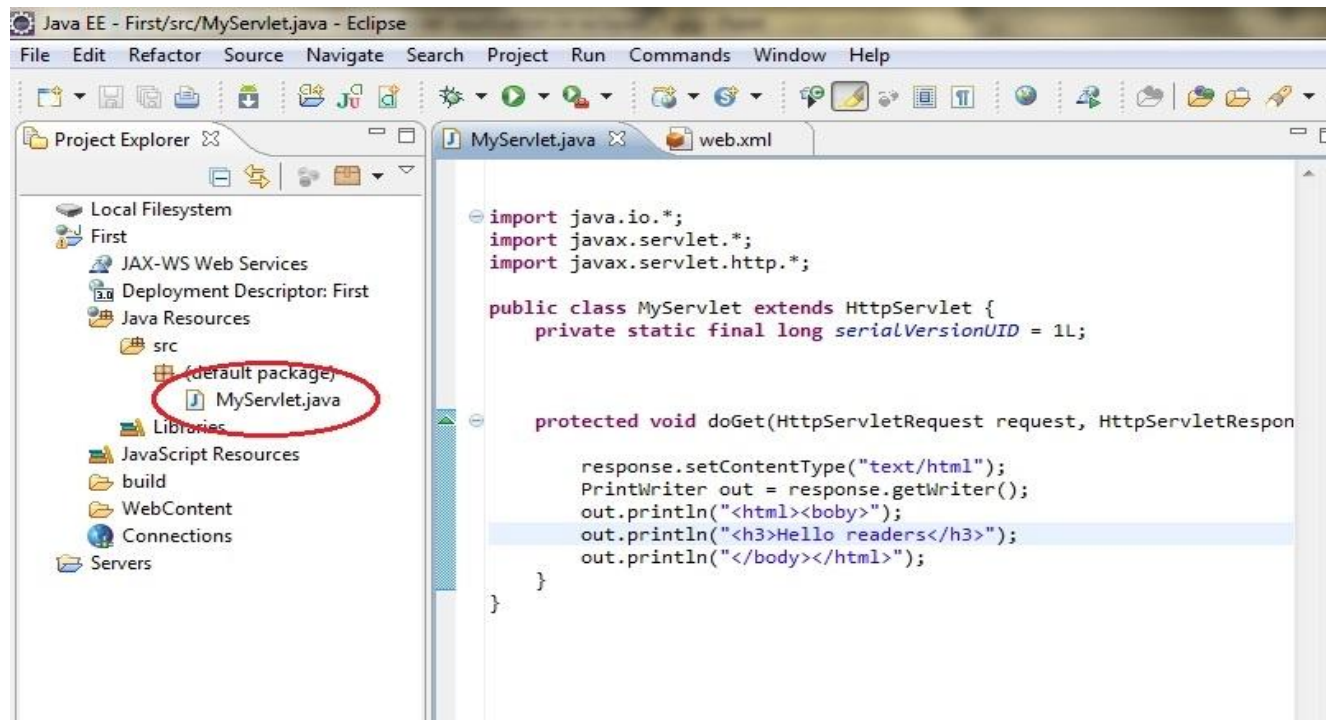
Using Eclipse IDE – Creating Servlet

- Click on First project, go to Java Resources -> src. Right click on src select New -> Servlet



Using Eclipse IDE – Creating Servlet

- Give Servlet class name and click **Next**
- Give your Servlet class a Name of your choice.
- Leave everything else to default and click **Finish**



Structure of a simple Servlet

Packages need to be Imported

Method used to process the request

Code to set text format and PrintWriter object is created to send data to the client

Methods used to process the GET and POST request

```
package example1;
import java.io.*;
import java.net.*;
import javax.servlet.*;
import javax.servlet.http.*;

public class HelloWorld extends HttpServlet {
    protected void processRequest(HttpServletRequest request, HttpServletResponse response) throws ServletException, IOException {
        response.setContentType("text/html;charset=UTF-8");
        PrintWriter out = response.getWriter();
        out.println("<html>");
        out.println("<head>");
        out.println("<title>Servlet HelloWorld</title>");
        out.println("</head>");
        out.println("<body>");
        out.println("<h1>Servlet HelloWorld at" + request.getContextPath() + "</h1>");
        out.println("</body>");
        out.println("</html>");
        out.close();
    }

    protected void doGet(HttpServletRequest request, HttpServletResponse response) throws ServletException, IOException {
        processRequest(request, response);
    }

    protected void doPost(HttpServletRequest request, HttpServletResponse response) throws ServletException, IOException {
        processRequest(request, response);
    }
}
```

Demonstration: Example 1

Servlets - Annotations

So far, you have learnt how Servlet uses the deployment descriptor (web.xml file) for deploying your application into a web server.

Servlet API 3.0 has introduced a new package called `javax.servlet.annotation`.

It provides annotation types which can be used for annotating a servlet class.

If you use annotation, then the deployment descriptor (web.xml) is not required.

But you should use tomcat7 or any later version of tomcat.

Servlets - Annotations

Annotations can replace equivalent XML configuration in the web deployment descriptor file (web.xml)

Such as servlet declaration and servlet mapping.

Servlet containers will process the annotated classes at deployment time.

Sr.No.	Annotation & Description
1	@WebServlet - To declare a servlet.
2	@WebInitParam - To specify an initialization parameter.
3	@WebFilter - To declare a servlet filter.
4	@WebListener - To declare a WebListener
5	@HandlesTypes - To declare the class types that a ServletContainerInitializer can handle.
6	@HttpConstraint This annotation is used within the ServletSecurity annotation to represent the security constraints to be applied to all HTTP protocol methods for which a corresponding HttpMethodConstraint element does NOT occur within the ServletSecurity annotation.
7	@HttpMethodConstraint This annotation is used within the ServletSecurity annotation to represent security constraints on specific HTTP protocol messages.
8	@MultipartConfig Annotation that may be specified on a Servlet class, indicating that instances of the Servlet expect requests that conform to the multipart/form-data MIME type.
9	@ServletSecurity This annotation is used on a Servlet implementation class to specify security constraints to be enforced by a Servlet container on HTTP protocol messages.

Servlet with Annotation – Servlet v3

- Annotation represents the metadata.
- If you use annotation, deployment descriptor (web.xml file) is not required.
- But you should have tomcat7 as it will not run in the previous versions of tomcat.
- **@WebServlet** annotation is used to map the servlet with the specified name.

```
@WebServlet("/Simple")  
public class Simple extends HttpServlet {  
}
```

Creating a Servlet

The servlet can be created by three ways:

- By implementing Servlet interface,
 - By inheriting GenericServlet class, (or)
 - By inheriting HttpServlet class
-
- The mostly used approach is by extending HttpServlet because it provides http request specific method such as doGet(), doPost(), doHead() etc.

GenericServlet class

- GenericServlet class implements Servlet, ServletConfig and Serializable interfaces.
- It provides the implementation of all the methods of these interfaces except the service method.
- GenericServlet class can handle any type of request so it is protocol-independent.
- You may create a generic servlet by inheriting the GenericServlet class and providing the implementation of the service method.

GenericServlet Class

- GenericServlet is an abstract class that provides implementation of most of the basic servlet methods. This is a very important class.
 - `public void init(ServletConfig)`
 - `public abstract void service(ServletRequest request, ServletResponse response)`
 - `public void destroy()`
 - `public ServletConfig getServletConfig()`
 - `public String getServletInfo()`
 - `public ServletContext getServletContext()`
 - `public String getInitParameter(String name)`
 - `public Enumeration getInitParameterNames()`
 - `public String getServletName()`
 - `public void log(String msg)`
 - `public void log(String msg, Throwable t)`

Example – Generic Servlet

```
import java.io.*;
import javax.servlet.*;

public class First extends GenericServlet{

    public void service(ServletRequest req,ServletResponse res)
    throws IOException,ServletException{

        res.setContentType("text/html");

        PrintWriter out=res.getWriter();
        out.print("<html><body>");
        out.print("<b>hello generic servlet</b>");
        out.print("</body></html>");
    }
}
```

Generic vs HTTP Servlet

- GenericServlet defines a generic, protocol-independent **servlet**.
- GenericServlet gives a blueprint and makes writing servlet easier.
- GenericServlet provides simple versions of the lifecycle methods `init` and `destroy` and of the methods in the `ServletConfig` interface.
- To write a generic servlet, it is sufficient to override the abstract service method.
- `HttpServlet` defines a HTTP protocol specific servlet.
- `HttpServlet` gives a blueprint for `Http` servlet and makes writing them easier.
- `HttpServlet` extends the `GenericServlet` and hence inherits the properties `GenericServlet`.

Servlet Interface

- Servlet interface provides common behaviour to all the servlets.
- Servlet interface defines methods that all servlets must implement.
- Servlet interface needs to be implemented for creating any servlet (either directly or indirectly).
- It provides 3 life cycle methods that are used to
 - Initialize the servlet,
 - To service the requests,
 - Destroy the servlet and 2 non-life cycle methods.

Methods of Servlet interface

Method	Description
public void init(ServletConfig config)	Initializes the servlet. It is the life cycle method of servlet and invoked by the web container only once.
public void service(ServletRequest request, ServletResponse response)	Provides response for the incoming request. It is invoked at each request by the web container.
public void destroy()	Invoked only once and indicates that servlet is being destroyed.
public ServletConfig getServletConfig()	returns the object of ServletConfig.
public String getServletInfo()	returns information about servlet such as writer, copyright, version etc.

Servlet by implementing Servlet interface

```
public class Second implements Servlet{  
    ServletConfig config=null;
```

```
    public void init(ServletConfig config){  
        this.config=config;  
        System.out.println("servlet is initialized");  
    }
```

```
    public void service(ServletRequest  
        req,ServletResponse res)  
        throws IOException,ServletException{
```

```
        res.setContentType("text/html");
```

```
        PrintWriter out=res.getWriter();  
        out.print("<html><body>");  
        out.print("<b>hello simple servlet</b>");  
        out.print("</body></html>");  
    }
```

```
    public void destroy(){  
        System.out.println("servlet is destroyed");  
    }
```

```
    public ServletConfig getServletConfig(){  
  
        return config;}  
}
```

```
    public String getServletInfo()  
    {  
        return "copyright 2010-2019";  
    }  
}
```

Summary 2-1

- Servlets allow the users to run Java code on the server and generate dynamic content.
- An `HTTP Servlet` sends `HTTP Request` and gets response in the form of `HTTP Response`.
- The various `HTTP methods` are `GET`, `POST`, `OPTIONS` and `DELETE`.
- The Servlet life cycle consists of three methods: `init()`, `service()` and `destroy()`.
- The Servlet API is contained in two packages: `javax.servlet` and `javax.servlet.http`.

Summary 2-2

- The Servlets implement interfaces by extending either from `GenericServlet` class or `HttpServlet` class.
- The Eclipse / NetBeans software is used for developing Java based applications, such as applets, servlets and JSPs.